University of BRISTOL
Centre for Academic Language and Development

Foundations of Computer Programming - LANG00052

Week 01

Intro to OOP

Syed Murad

Learning outcomes

- Understand programming and its fundamental concepts.
- Understand different programming languages and recognise the difference
- Understand characteristics of High-level and Low-level Programming Languages
- Understand the Rules of a Language (Syntax & Semantics)
- Develop a Comprehensive Understanding of Object-Oriented Programming
- Recognise the distinction between Procedural and Object-Oriented Programming approaches.
- Understand the fundamental principles of object-oriented programming

What is programming?

Programming is the process of creating a set of instructions that tell a computer how to perform a task. Programming can be done using a variety of computer programming languages









Programming is just a tool to help you what you want to do

High level and Low level

```
11010100 0011 11001101
01011100 1010 10001111
11001111 1010 11111110
10000111 1011 11000001
```

```
mov ecx, ebx
mov esp, edx
mov edx, r9d
mov rax, rdx
```



Benefits of using OOP Approach

- Re-usability
- Extendibility
- Timely delivery (speed)
- Maintainability
- Reliability

Object Oriented Programming

What is Object Oriented Programming?
Object-oriented programming (OOP) is a computer programming model that organises software design around data, or objects, rather than functions and logic. An object can be defined as a data field that has unique attributes and behaviour.

Explain the concepts of Object and Class in OOP
An object is a self-contained entity which has state and behaviour. It may have properties and attributes which define its state and methods which define its behaviour.

A class is the blueprint from which objects can be created. The class is the model which holds the attributes and methods of the objects that would be created from the class.

What is an object?

You!

Car

Book

Also

Your GP appointment

Bank account



Object

You!

What is your properties?
What you have?
What can you do?



Do you have a bank account?

What is a Class?
To create object, you need class…

Attribute/Properties/State

Phone
Model
Colour
Weight
Size
…

Bank account
Account number
Account name
…



Operations/Methods/Behaviour

Phone
Dialling a number
Make call
End call
…

Bank account
Show balance
Depositing money
…



Group of objects that have some common attributes and behaviour.

Class?

Person



Class and Object





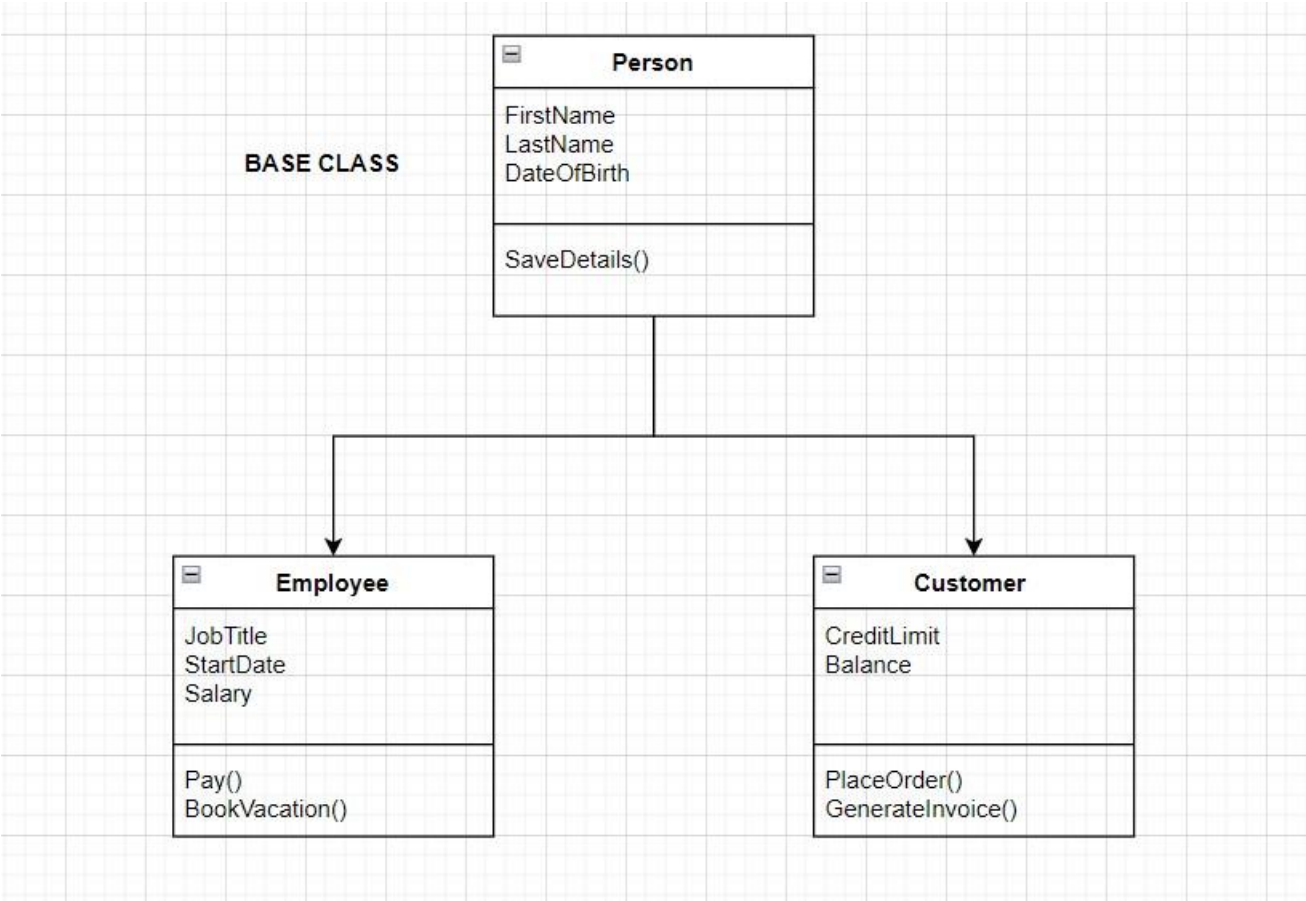








Each object is an instance of a class in the computer’s memory


BASE CLASS
Person
FirstName
LastName
DateOfBirth
SaveDetails()
Employee
JobTitle
StartDate
Salary
Pay()
BookVacation()
Customer
CreditLimit
Balance
PlaceOrder()
GenerateInvoice()

THE FOUR FUNDAMENTAL CONCEPTS OF OBJECT-ORIENTED PROGRAMMING.

1. Abstraction
2. Inheritance
3. Encapsulation
4. Polymorphism





To simply reality

Abstraction

Abstraction in OOP refers to an approach in programming where we aim to show important information and hide unnecessary details. It allows different objects from different location to interact with each other and use each others' properties without really requiring to know the internal structure of the other object.

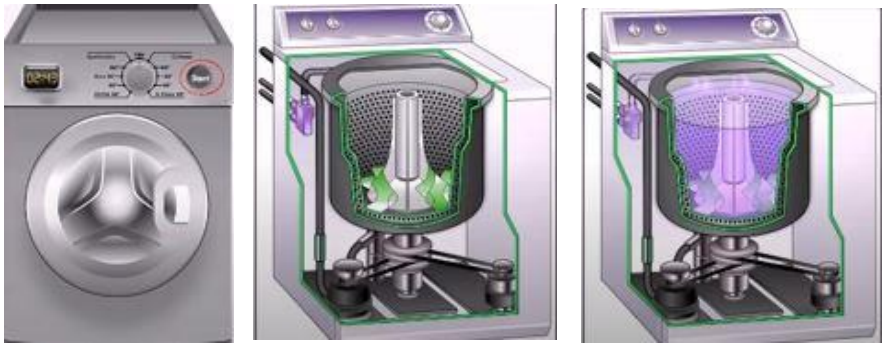


Encapsulation

Encapsulation is the packaging of Objects and variables of a particular kind so that its content can be protected or made accessible to external entities



Hide data and complexity (information hiding)

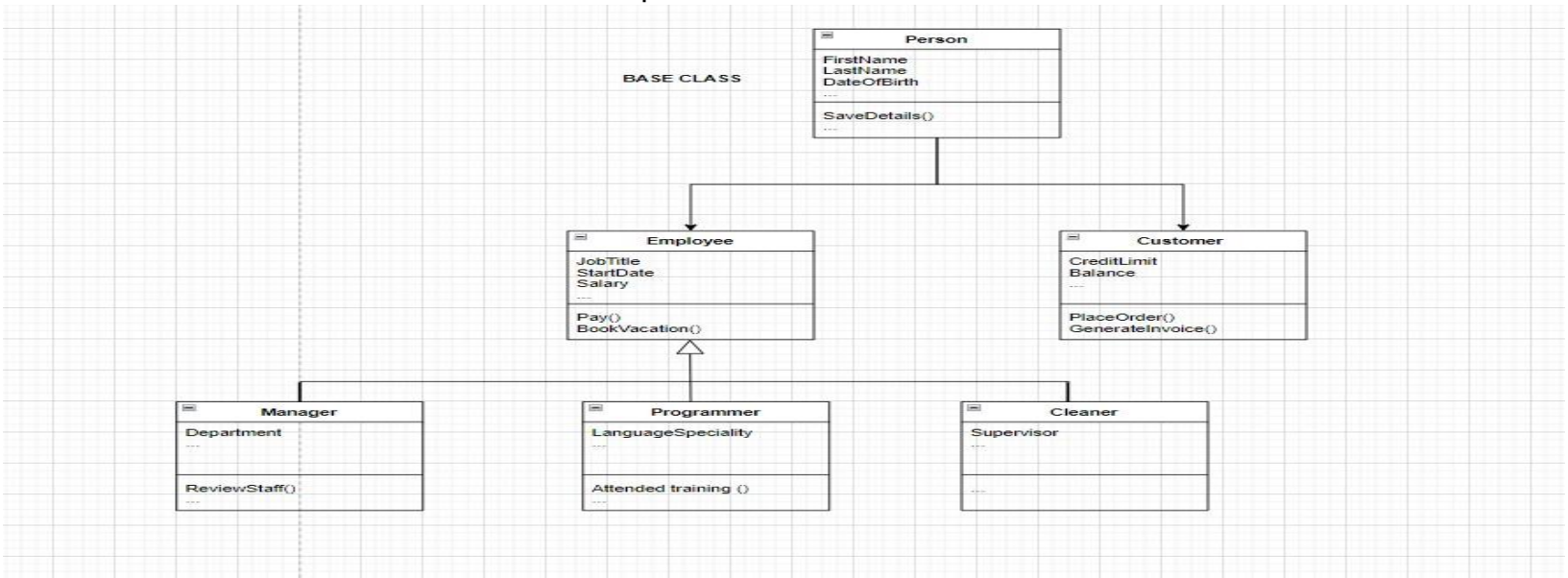




Inheritance

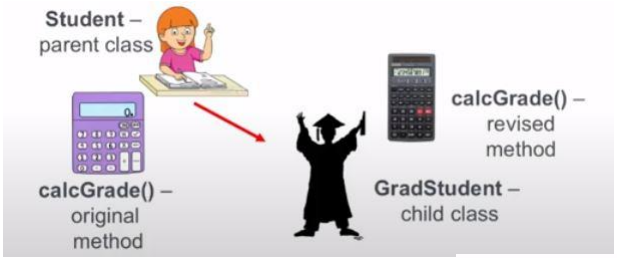
In OOS objects can inherit the behaviour or properties of another object and build upon it.

Super class and Sub class



Polymorphism

Polymorphism allows an object to exist in different forms. It gives programming languages the ability to redefine methods for derived classes and use them for different specific purposes.





A class can implement an inherited method in its own way

Syntax & Semantics

Syntax
The set of rules that define the structure of valid statements in a programming language.

Correct Syntax: x = 5;
Incorrect Syntax: x 5 =

Semantics
The meaning behind the code or what the statements do when executed.

x = "5" + 5 # Syntax is correct, but this causes a semantic error (type mismatch).

Syntax VS Semantics

Syntax:

- How code is written.
- Correct syntax ensures code compiles/runs.

Semantics:

- What the code means or does.
- Correct semantics ensures code does what is intended.

Brief History of Python

- Invented in the Netherlands, early 90s by Guido van Rossum
- Named after Monty Python
- Open sourced from the beginning
- Considered a scripting language, but is much more
- Scalable, object oriented and functional from the beginning

Time for questions